

Timing-Feedback Adaptive Scheduler for Split-6 Latency Reduction

Ming-Hong HSU*, Ray-Guang Cheng*, Co-author 1[†]

* Dept. of Electronic and Computer Engineering, National Taiwan University of Science and Technology, Taiwan
Email: crg@mail.ntust.edu.tw

Abstract Section (before Introduction) Add a clear placeholder for an abstract:

Purpose: Briefly summarize the problem, method, results, and contribution.

Word count: typically 150—250 words.

Keywords: list 3—6 keywords.

I. Introduction

Directly reusing the FAPI slot.indication mechanism in nFAPI mode without following the SCF specification leads to significant latency because FAPI timing control was not designed for nFAPI's transport and synchronization characteristics. Automated testing and analysis are required to detect and verify the timing delays caused by non-compliant implementations with the SCF specification.

Each step of the nFAPI procedure must be individually measured and analyzed to isolate delay sources through repeated end-to-end testing. Continuous automated testing, data collection, and graphical analysis are needed to compare delay patterns and identify timing bottlenecks efficiently.

A. Related Works

Purpose: To show how your work fits into the existing body of knowledge, and how it differs from or improves upon previous work.

Typical content:

- Summary of key prior research
- Strengths and limitations of existing approaches
- Comparison to your approach
- Justification for why a new approach is needed

Example phrases:

- "Several studies have explored ..."
- "Unlike [Author], we propose ..."
- "Prior work has not considered ..."

The main contributions of this paper are summarized as follows.

- A fully SCF-compliant nFAPI implementation flow is established, replacing the legacy slot.indication mechanism with a delay-management process and introducing a VNF-side timing-window maintenance algorithm to cover undefined timing logic in the standard.
- An SMO RAPP module is developed to automate end-to-end testing, data acquisition, and chart generation, enabling validation and optimization of the proposed delay-management mechanism.

The rest of the paper is organized as follows. Section II defines the system model/architecture considered in this paper. The proposed analytical model/method is given in Section III. Section IV shows the numerical results. Concluding remarks are given in Section VI.

II. System Model

Figure 1 illustrates the system architecture of the proposed Timing-Feedback Adaptive Scheduler within a Split-6 RAN environment. The system comprises two main entities: the Virtual Network Function (VNF) and the Physical Network Function (PNF), connected via a packet-switched network.

The VNF hosts the upper layer protocol stack and the scheduler. To minimize transmission latency and ensure synchronization with the PNF, the VNF implements a feedback-based timing mechanism. The key components include:

- **Timing-Feedback Adaptive Module:** This module acts as the core controller. It receives timing feedback from the PNF, specifically the difference between the packet arrival time and the expected slot time (denoted as late by $N \mu s$ or early by $M \mu s$). It also incorporates internal scheduler timing metrics. Based on these inputs, it dynamically computes the optimal sleep duration for the VNF timing thread to align with the PNF's slot cycle.
- **VNF Timing Thread:** This thread is responsible for triggering the scheduler instances. It utilizes the corrected timing provided by the Adaptive Module to wake up at the precise moment, compensating for network jitter and processing delays.
- **Scheduler:** Detailed scheduling operations are performed here. Multiple scheduler instances generate the Transmission Time Interval (TTI) messages, which are then transmitted to the PNF.
- **PNF:** The Physical Network Function receives the TTI messages for radio transmission. It measures the arrival time of these messages and feeds this "Timing Info" back to the VNF, closing the control loop.

The primary objective of this system is to adaptively adjust the VNF's execution offset such that TTI messages arrive at the PNF within the valid reception window, thereby avoiding slot drops due to late arrival while minimizing buffering latency caused by excessive earliness.

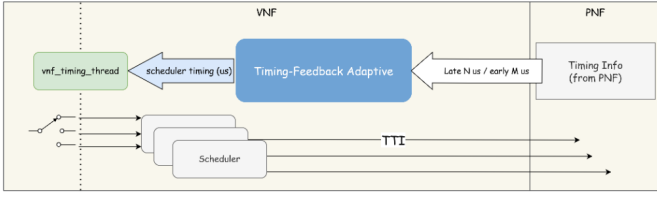


Fig. 1. System Model of the Timing-Feedback Adaptive Scheduler.

III. Proposed Method

To address the timing synchronization challenges in Split-6 RAN, we propose a Timing-Feedback Adaptive Scheduler. This mechanism introduces a closed-loop control system that dynamically adjusts the VNF's execution timing based on feedback from the PNF. As shown in Section II, the core objective is to minimize the timing difference between the arrival of TTI messages and the PNF's slot boundary, ensuring reliable transmission while minimizing latency. The proposed method consists of three key components: Jitter Mitigation via Baseline Envelope, Drift Prevention with a Time Bank mechanism, and Severe Deviation Handling.

A. Dynamic Timing Adjustment Algorithm

The core of the proposed method is an adaptive algorithm that calculates the optimal sleep duration (T_{sleep}) for the VNF timing thread. Let $t_{arrival}$ be the arrival time of the TTI message at the PNF and t_{slot} be the start time of the corresponding slot. The timing error Δt is defined as:

B. VNF-PNF Clock Synchronization

To achieve precise time alignment between the VNF and PNF, a four-way handshake mechanism (Node Sync) is employed. Let t_1 be the timestamp when the VNF sends a synchronization request, t_2 and t_3 be the timestamps when the PNF receives and responds to the request, and t_4 be the timestamp when the VNF receives the response. The clock offset Δt_{offset} and the one-way delay T_{owd} are calculated as follows:

$$\Delta t_{offset} = \frac{(t_2 - t_1) - (t_4 - t_3)}{2} \quad (1)$$

$$T_{owd} = \frac{(t_4 - t_1) - (t_3 - t_2)}{2} \quad (2)$$

A positive Δt_{offset} indicates that the VNF clock is lagging behind the PNF, while a negative value indicates it is ahead. To maintain a stable buffer at the PNF, a target margin T_{margin} is added to the adjustment. The resulting slot-level adjustment N_{slot_adj} and microsecond-level adjustment T_{us_adj} are:

$$N_{slot_adj} = \lfloor \frac{\Delta t_{offset} + T_{margin}}{T_{slot}} \rfloor \quad (3)$$

$$T_{us_adj} = (\Delta t_{offset} + T_{margin}) \pmod{T_{slot}} \quad (4)$$

C. Jitter Mitigation via Baseline Envelope

Network jitter and processing variations can cause significant fluctuations in Δt . Direct reaction to every instantaneous value would lead to system instability. To mitigate this, we employ a Baseline Envelope mechanism. The system maintains a variable E_{base} representing the minimum observed processing and transmission delay.

- **Fast Attack:** If the current measured delay d_{curr} is less than E_{base} , we update $E_{base} = d_{curr}$. This allows the system to quickly adapt to improved network conditions.
- **Slow Decay:** If $d_{curr} > E_{base}$, we treat the excess as potential jitter. E_{base} is slowly incremented (decayed) over time to ensure it doesn't get stuck at a transient minimum.

This envelope provides a stable reference point, effectively filtering out high-frequency noise.

D. Drift Prevention with Time Bank Mechanism

Traditional Proportional-Integral (PI) controllers can suffer from integral windup or oscillation in this context. We propose a "Time Bank" mechanism, utilizing a variable $T_{pending}$, to manage accumulated timing deviation without over-correction.

- **Accumulation:** When the system is late ($\Delta t > 0$), the lateness is added to $T_{pending}$. This represents a debt of time that must be recovered.
- **Gradual Recovery:** Instead of forcing an immediate large correction, the scheduler pays back this debt only when it has "spare time" (i.e., when the estimated computation time allows).

This approach ensures that the recovery process does not induce new violations in subsequent slots.

E. Severe Deviation Handling

In cases of extreme network disruption or process preemption, the VNF may fall significantly behind (e.g., lagging by more than one entire slot duration T_{slot}). Attempting to process these stale slots is futile. We implement a slot-skipping mechanism:

$$N_{skip} = \lfloor \frac{\Delta t_{accumulated}}{T_{slot}} \rfloor \quad (5)$$

If $N_{skip} \geq 1$, the scheduler immediately advances its internal slot counter by N_{skip} and resets the timing state. This "hard reset" allows the system to re-synchronize instantly with the current PNF time, preventing a cascade of late slots.

IV. Numerical/Simulation/Experimental Results

Computer simulations/Experiments were conducted to verify xxx (the effectiveness of the proposed analytical model). In the following figures, each point represented the average value of 10^5 samples. In each sample, we collect the statistics of xxx. Symbols and lines were used to show the simulation and analytical results, respectively.

XXX scenarios were investigated. Scenario I was designed to demonstrate/verify the accuracy/effectiveness of Contribution 1.

A. Contribution 1

Fig. 1.1: Purpose X axis: Y axis:

B. Contribution 2

Fig. 2.1: X axis: Y axis:

C. Contribution 3

Fig. 3.1: X axis: Y axis:

References

[1]